Workflow Engine

How workflow state transitions are orchestrated

The workflow engine is a thin executor that reads the table, evaluates conditions, checks permissions, and fires actions. The engine code never changes when a state's workflow changes, only the data rows do.

Constraint

The engine is configurable but not infinitely configurable. Transitions, conditions, and actions marked `IsFederal = true` cannot be overridden by any tenant configuration. This is the guardrail that makes the engine safe so states can configure their workflow variations without being able to change federally mandated behavior.

Tables

WorkflowEntityTypes cnp_global
EntityTypeId PK
EntityTypeCode e.g. 'Claim', 'Application', 'AdminReview', 'DCMatch'
Description
ConditionEvaluator Class/service name the engine calls to evaluate conditions for this type
ActionExecutor Class/service name the engine calls to fire actions for this type

WorkflowStates cnp_global
StateId PK
EntityTypeId FK
StateCode 'Draft', 'Submitted', 'Approved', etc.
StateName Display label
IsFederal 🔒 True = cannot be removed or renamed by tenant config
IsTerminal True = no outbound transitions (Locked, Paid, Terminated)

DisplayColor UI badge color hex
DisplayOrder

WorkflowTransitions cnp_global
TransitionId PK
EntityTypeId FK
FromStateId FK
ToStateId FK
TransitionCode 'SubmitClaim', 'ApproveClaim', 'LockClaim'
IsFederal 🔒 True = cannot be disabled or overridden by tenant
IsAutomatic Fires without user action when conditions met
TriggerEvent 'OnUserAction', 'OnSave', 'OnSchedule', 'OnExternalEvent'
RequiresComment User must enter a reason to proceed
TransitionName Button label shown to user

WorkflowTransitionConditions cnp_global
ConditionId PK
TransitionId FK
ConditionCode Named hook: 'AllErrorsResolved', 'PastDueDate', 'HasReviewerRole'
ConditionType 'PreCondition' (must be true to allow) or 'PostCondition' (must be true after)
ConditionParams JSONB parameters passed to the named hook evaluator
IsFederal 🔒 True = cannot be removed by tenant override
FailureMessage User-facing message when condition not met

WorkflowTransitionActions cnp_global

ActionId PK
TransitionId FK
ActionCode Named hook: 'FreezeClaimLaws', 'CreateDisbursement', 'SendNotification'
ExecutionOrder Actions within a transition fire in this order
IsFederal 🔒 True = cannot be removed by tenant override
ActionParams JSONB parameters passed to the action executor
IsRollbackable True = if later action fails, this one can be undone

TenantTransitionOverrides cnp_ops
OverrideId PK
TransitionId FK
TenantCode
OverrideType 'Disable', 'MakeAutomatic', 'AddCondition', 'RemoveCondition', 'ChangeRole'
OverrideParams JSONB — e.g. which condition to add/remove, new role code
EnabledAt
EnabledBy WebUsers — who activated this override **FK**

Note: Engine rejects overrides where IsFederal=true on the target row

WorkflowRolePermissions

A join table between WorkflowTransitions and role codes: `PermissionId PK | TransitionId FK | RoleCode | TenantCode (null = all tenants)`. Defines who can trigger which transitions. When TenantCode is null, the permission applies platform-wide. A tenant can add more roles to a transition via TenantTransitionOverrides (OverrideType = 'AddRole'), but cannot remove a role that has `IsFederal = true`.

Entity Types

The `WorkflowEntityTypes` table defines every entity that can have a workflow state. Entity type determines which `ConditionEvaluator` and `ActionExecutor` are called.

`CACFPFinancialReview` is spawned by an action hook on the `AdminReview → FindingsDocumented` transition when financial findings are present. It is a separate workflow instance linked by `ParentEntityId`.

EntityTypeCode	Description	ConditionEvaluator	ActionExecutor	Programs
`Claim`	Monthly reimbursement claim	`ClaimConditionEvaluator`	`ClaimActionExecutor`	NSLP, SBP, SSO, CACFP, SFSP
`SponsorApp`	Sponsor annual application / renewal	`ApplicationConditionEvaluator`	`ApplicationActionExecutor`	All
`SiteApp`	Site-level application and renewal	`ApplicationConditionEvaluator`	`ApplicationActionExecutor`	All
`LateClaimException`	Authorization for claims past 60-day window	`LateClaimConditionEvaluator`	`LateClaimActionExecutor`	All
`AdminReview`	On-site and off-site administrative review	`ReviewConditionEvaluator`	`ReviewActionExecutor`	NSLP, SBP, CACFP, SFSP
`ProcurementReview`	3-year SFA purchasing compliance audit	`ReviewConditionEvaluator`	`ReviewActionExecutor`	NSLP, SBP
`CACFPFinancialReview`	Budget, admin expense cap, fiscal accountability review	`FinancialReviewConditionEvaluator`	`FinancialReviewActionExecutor`	CACFP

`CorrectiveAction`	Finding response, sponsor dialogue, resolution tracking	`CorrectiveActionConditionEvaluator`	`CorrectiveActionExecutor`	All
`Appeal`	Sponsor challenge to adverse decisions	`AppealConditionEvaluator`	`AppealActionExecutor`	All
`WaiverRequest`	Federal/state waiver submission and tracking	`WaiverConditionEvaluator`	`WaiverActionExecutor`	All
`CapExpRequest`	Capital equipment purchase approval above threshold	`CapExpConditionEvaluator`	`CapExpActionExecutor`	CACFP

Federal Guardrail Mechanism

This table defines what IsFederal protects at each layer.

Layer	Protection
WorkflowStates.IsFederal = true	'Approved', 'Locked', 'Paid' on Claims cannot be renamed or deleted. A state cannot remove the Locked status from the claim lifecycle.
WorkflowTransitions.IsFederal = true	Approved→Locked on Claims cannot be disabled. A state cannot remove the lock step from the claim approval chain.
WorkflowTransitionConditions.IsFederal = true	'AllClaimErrorsResolved' on ApproveClaim cannot be removed. A state cannot approve a claim with unresolved validation errors.
WorkflowTransitionActions.IsFederal = true	'FreezeClaimLaws' on Approved→Locked cannot be removed. A state cannot approve a

	claim without freezing the sponsor's eligibility tier record (rate tier, meal groups, operating parameters).

Engine Enforcement

The enforcement is at write time, not read time. When a `TenantTransitionOverride` row is written, the engine performs a guardrail check before the INSERT commits:

```
-- Pseudo-code for the guardrail check in the override write service\nIF override.OverrideType = 'Disable'\n  AND WorkflowTransitions[override.TransitionId].IsFederal = true\n  THEN RAISE GuardrailViolationException('Cannot disable a federally required transition')\n\nIF override.OverrideType = 'RemoveCondition'\n  AND WorkflowTransitionConditions[override.ConditionId].IsFederal = true\n  THEN RAISE GuardrailViolationException('Cannot remove a federally required condition')
```

IsFederal is not user-configurable. The `IsFederal` flag is set at system deployment and can only be changed via a versioned database migration reviewed by a senior engineer. It is never exposed in the admin UI. A state agency administrator cannot see or change it.

Database Enforcement

Application-layer checks fire only when the write passes through the API. A direct `INSERT INTO TenantTransitionOverrides` from a SQL client, a data-fix migration, or a CI/CD deployment script bypasses write service guardrails. An additional level of security is added for Federal compliance.

Trigger

Two BEFORE INSERT OR UPDATE triggers on the override tables enforce the same guardrail logic at the database layer. They fire for every write regardless of source.

trg_block_federal_transition_override on `TenantTransitionOverrides` fires BEFORE INSERT OR UPDATE.

```
CREATE OR REPLACE TRIGGER trg_block_federal_transition_override\nBEFORE INSERT OR UPDATE ON TenantTransitionOverrides\nFOR EACH ROW\nBEGIN\n  IF NEW.OverrideType IN ('AddCondition', 'ReplaceCondition') THEN\n    IF EXISTS (SELECT 1 FROM WorkflowTransitionConditions WHERE ConditionCode = NEW.ConditionCode AND IsFederal = true) THEN\n      PERFORM log_federal_guardrail_violation('TenantTransitionOverrides', NEW);\n      RAISE EXCEPTION 'Federal guardrail violation: condition % is federally required. Override rejected.', NEW.ConditionCode;\n    END IF;\n  END IF;\n  IF NEW.OverrideType IN ('AddAction', 'ReplaceAction') THEN\n    IF EXISTS (SELECT 1 FROM WorkflowTransitionActions WHERE ActionCode = NEW.ActionCode AND IsFederal = true) THEN\n      PERFORM log_federal_guardrail_violation('TenantTransitionOverrides', NEW);\n      RAISE EXCEPTION 'Federal guardrail violation: action % is federally required. Override rejected.', NEW.ActionCode;\n    END IF;\n  END IF;\n  IF NEW.OverrideType = 'RemoveCondition' THEN\n  IF EXISTS (SELECT 1 FROM WorkflowTransitionConditions WHERE ConditionCode = NEW.ConditionCode AND IsFederal = true) THEN\n      PERFORM log_federal_guardrail_violation('TenantTransitionOverrides', NEW);\n      RAISE EXCEPTION 'Federal guardrail violation: condition % cannot be removed — it is federally required.', NEW.ConditionCode;\n    END IF;\n  END IF;\n  IF NEW.OverrideType = 'RemoveAction' THEN\n
```

```
IF EXISTS (SELECT 1 FROM WorkflowTransitionActions WHERE ActionCode = NEW.ActionCode AND IsFederal = true) THEN\n      PERFORM log_federal_guardrail_violation('TenantTransitionOverrides', NEW);\n      RAISE EXCEPTION 'Federal guardrail violation: action % cannot be removed — it is federally required.', NEW.ActionCode;\n    END IF;\n  END IF;\n  IF EXISTS (SELECT 1 FROM WorkflowTransitions t JOIN WorkflowStates s ON s.StateId IN (t.FromStateId, t.ToStateId) WHERE t.TransitionId = NEW.TransitionId AND s.IsFederal = true) THEN\n    PERFORM log_federal_guardrail_violation('TenantTransitionOverrides', NEW);\n    RAISE EXCEPTION 'Federal guardrail violation: transition % involves a federally protected state. Override rejected.', NEW.TransitionId;\n  END IF;\nEND;
```

trg_block_federal_config_override on `TenantConfigOverrides` fires BEFORE INSERT OR UPDATE.

```
CREATE OR REPLACE TRIGGER trg_block_federal_config_override\nBEFORE INSERT OR UPDATE ON TenantConfigOverrides\nFOR EACH ROW\nBEGIN\n  IF EXISTS (SELECT 1 FROM ConfigDefinitions WHERE ConfigKey = NEW.ConfigKey AND IsFederal = true) THEN\n    PERFORM log_federal_guardrail_violation('TenantConfigOverrides', NEW);\n    RAISE EXCEPTION 'Federal guardrail violation: config key % is federally locked. Override rejected.', NEW.ConfigKey;\n  END IF;\nEND;
```

Database role separation

Role	Permissions	Used by
`cnp_app`	INSERT / UPDATE / DELETE on `TenantTransitionOverrides`, `TenantConfigOverrides`, and other operational tables. **Read-only** on `WorkflowStates`, `WorkflowTransitionConditions`, `WorkflowTransitionActions`, `ConfigDefinitions`.	Application service accounts. The tables the triggers read are migration-managed — the app role cannot alter them to bypass trigger logic.
`cnp_migration`	Can modify schema tables. Cannot be used interactively.	CI/CD pipeline only, during versioned deployments. Not available to engineers as a connection credential.
`cnp_readonly`	No INSERT / UPDATE / DELETE anywhere.	Offshore engineers in dev/staging for debugging and query work.

DBA (production)	Read-only by default. Write access requires a break-glass procedure that generates a timestamped audit event before the elevated session is opened.	Production incidents only. The break-glass event is visible to the security team in real time.

Example

Write source	Application guardrail	Database trigger
API call through write service	Yes	Yes
Direct SQL via SQL client (DBA, engineer)	No	Yes
Migration script run during deployment	No	Yes
Runbook or data-fix script	No	Yes
ORM or query builder bypassing service layer	No	Yes

Audit integration

The `log_federal_guardrail_violation()` helper called inside each trigger writes a row to `AuditLogs` before the exception propagates. The INSERT into `AuditLogs` runs in an autonomous transaction so that the rollback of the rejected write does not erase the audit record.

EntityType = 'FederalGuardrailViolation'

Every blocked attempt, regardless of source, produces an `AuditLogs` row with `EntityType = 'FederalGuardrailViolation'`, the source role, the targeted table, the rejected row values, and a timestamp.

State Machine: Claims

Claim lifecycle state machine (red border = IsFederal)

Claim WorkflowTransitions data rows

TransitionCode	From → To	IsFederal	IsAutomatic	Key Conditions	Key Actions
SubmitClaim	Draft → Submitted	No	No	AllMealCountsPresent, HasSubmitterRole	CreateAuditLog, TriggerValidation
HoldForError	Submitted → HeldForError	Yes	Yes (OnSave)	ValidationErrorsExist	CreateAuditLog, NotifySubmitter
ResubmitClaim	HeldForError → Submitted	No	No	AllErrorsResolved, HasSubmitterRole	CreateAuditLog, TriggerValidation
BeginReview	Submitted → UnderReview	Yes	Yes (OnSave)	NoValidationErrors	CreateAuditLog, AssignReviewer
ApproveClaim	UnderReview → Approved	Yes	No	AllErrorsResolved, HasReviewerRole, 🔒 federal	CreateAuditLog, CreateEntitySnapshot
LockClaim	Approved → Locked	Yes	Yes (OnApprove)	—	🔒 FreezeClaimLaws, LockRecord, CreateEntitySnapshot
DisburseClaim	Locked → Paid	Yes	Yes (OnDisbursement)	DisbursementCreated	CreateAuditLog
VoidClaim	UnderReview → Void	No	No	HasReviewerRole,	CreateAuditLog,

				RequiresComment	NotifySubmitter

Example: Maine tenant override to auto-decline late claims

Maine's nTIG contract requires automatic declination of claims submitted more than 60 days past the program month end. This is a state policy, not a federal requirement. The override:

```
-- TenantTransitionOverrides row for Maine\nTransitionId  → SubmitClaim (Draft→Submitted)\nTenantCode    → 'ME'\nOverrideType  → 'AddCondition'\nOverrideParams → {\n "conditionCode": "NotPastStateLateClaimCutoff",\n  "onFailAction": "AutoDecline",\n "cutoffDays": 60\n}
```

The base SubmitClaim transition is unchanged for all other states. Maine's condition adds to it. The guardrail check passes because SubmitClaim.IsFederal = false.

State Machine: Applications

Application lifecycle state machine

The **Prospect** state is a DIS-specific capability (PotentialSponsor tracking) that is absent from base CTS. It is not federal (`IsFederal = false`), so it can be present in DIS-origin tenants and absent from CTS-origin tenants via TenantTransitionOverrides without affecting the federal compliance chain. **PendingDocs** is similarly state-configurable. The Approve and Deny transitions out of UnderReview are federal and cannot be skipped or renamed.

Additional State Machines

Federal actions highlighted in blue. Other actions are optional.

AdminReview

Scheduled → In Progress → FindingsDocumented → SupervisorReview → CorrectiveActionRequired → AppealWindow → **Closed**

TransitionCode	From → To	IsFederal	Key Conditions	Key Actions
`BeginReview`	Scheduled → InProgress	**No**	HasReviewerRole, ReviewDateReached	AssignReviewer, CreateAuditLog
`DocumentFindings`	InProgress → FindingsDocume	**Yes**	AllFindingsEntered,	CreateEntitySnapshot,

	nted		HasReviewerRole	SpawnCorrectiveActions
`RequireCorrectiveAction`	FindingsDocumented → CorrectiveActionRequired	**Yes**	HasSeriousFindings	SpawnCorrectiveActionWorkflows, SendNotification
`CloseClean`	FindingsDocumented → Closed	**Yes**	NoSeriousFindings	CreateEntitySnapshot, SendNotification
`CloseResolved`	CorrectiveActionRequired → Closed	**Yes**	AllCorrectiveActionsClosed	CreateEntitySnapshot, SendNotification

Corrective Action

Open → PendingResponse → UnderStateReview → Resolved → **Closed**

Closed triggers `IsFiscalActionRequired` check. If true, `FiscalActionHook` creates a repayment schedule in the Accounting domain.

Appeal

Filed → Informal Resolution → Formal Hearing → DecisionIssued → **Closed**

DecisionIssued may trigger reversal of the parent entity state (e.g., reinstating a Denied application) via the `ApplyAppealDecision` action hook.

LateClaimException

Default → Submitted → PendingApproval → Approved → **Linked**

Approved triggers `LinkExceptionToClaim` — reopens the rejected claim for resubmission. Some states add a director approval step via override.

Named Hooks: Conditions and Actions

Conditions and actions are named references to typed functions implemented in code. The database row stores the name; the engine resolves it to the implementation. This is the key guardrail against encoding federal math in configuration.

Condition hooks — examples

ConditionCode	What it checks	IsFederal	Params (JSONB)
`AllClaimErrorsResolved`	ClaimErrors table has no rows with BrokenInd=true for this claim	Yes	—
`HasRole`	The requesting user has the specified role for this tenant	No	`{"role":"StateReviewer"}`
`NotPastDueDate`	Today is on or before the claim period's due date	No	`{"offsetDays":0}`
`PastLateClaimCutoff`	Today exceeds N days past the claim period end	No	`{"cutoffDays":60}`
`AllRequiredDocsAttached`	All required document types for this application type are attached	No	—
`NoOpenSeriousDeficiency`	Sponsor has no active SerDef proceeding	Yes	—
`ISPMeetsThreshold`	CEP group ISP ≥ 40% (for CEP election transition)	Yes	—
`VCAChecklistPassed`	VCA checklist score meets approval threshold	Yes	—

Action hooks — examples

ActionCode	What it does	IsFederal	Params (JSONB)
`FreezeClaimLaws`	Runs ClaimLaws.FreezeOnApproval() — locks the	Yes	—

	sponsor's **eligibility tier** (FedRateLevelCode, MealGroupCodes, operating parameters) into a ClaimLaws record. Dollar amounts are **NOT** captured here — they resolve from ProgramMealRates at claim compute time, keyed by (RateTypeCode, ClaimDate). This separation is what makes annual rate updates a data-only operation with no reprocessing of existing approvals.		
`LockRecord`	Sets IsLocked=true on the entity; blocks all further field edits	Yes	—
`CreateEntitySnapshot`	Writes a row to EntitySnapshots with current field values as JSONB	Yes	`{"fields": ["all"]}`
`SendNotification`	Writes to NotificationQueue with specified template	No	`{"templateCode": "ApplicationApproved"}`
`AssignReviewer`	Runs reviewer assignment logic	No	—

	based on program, state, and workload		
`TriggerValidation`	Runs the claim validation service, writes ClaimErrors rows	Yes	—
`CreateDisbursementRecord`	Creates a pending Disbursements row linked to the approved claim	Yes	—
`AutoDecline`	Fires DenyClaim transition automatically with system-generated comment	No	`{"reason":"PastLateClaimCutoff"}`

data drives WHICH hooks run; code defines WHAT they do. Adding a new state to the claim lifecycle requires a data row in WorkflowStates. Adding a new condition requires both a data row in WorkflowTransitionConditions AND a code implementation of the named hook. This is intentional. Configuration is fast; extending the hook library requires review.

Document Gates Condition Pattern

Document requirements are encoded as `WorkflowTransitionConditions` rows with `ConditionCode = 'AllRequiredDocsAttached'`. The `ConditionParams` JSONB specifies which document types are required. No separate table needed.

Federal vs. state document requirements: Conditions with `IsFederal = true` cannot be removed by any TenantTransitionOverride. State-specific document requirements have `IsFederal = false` and can be added or removed per tenant by PS.

ConditionParams key	Type	Description
`requiredTypes`	string[]	Document type codes that must be attached and in Accepted status

`anyOf`	string[]	At least one of these types must be present
`minCount`	int	Minimum number of attached documents regardless of type

```
-- Platform row: federal document requirement on SponsorApp (IsFederal = true — cannot be removed)\nWorkflowTransitionConditions:\n  TransitionId   = [SubmitSponsorApp: Draft -> Submitted]\n  ConditionCode  = 'AllRequiredDocsAttached'\n  IsFederal      = true\n  ConditionParams = { "requiredTypes": ["FreeReducedPricePolicyStatement", "SignedAgreement"] }\n  FailureMessage = "Free and Reduced Price Policy Statement and signed agreement required by 7 CFR 210.9."\n\n-- Tenant override: Alaska adds a state-specific affidavit requirement (IsFederal = false)\nTenantTransitionOverrides:\n  TransitionId   = [ApproveSiteApp: UnderReview -> Approved]\n  TenantCode     = 'AK'\n  OverrideType   = 'AddCondition'\n  OverrideParams = {\n    "conditionCode": "AllRequiredDocsAttached",\n    "conditionParams": { "requiredTypes": ["AKSiteAffidavit"] },\n    "failureMessage": "Alaska site affidavit must be attached before approval."\n  }
```

Notification Rules: Action Pattern

Notifications are `WorkflowTransitionActions` rows with `ActionCode = 'SendNotification'`. Multiple notification actions on one transition use different `ExecutionOrder` values. Delayed notifications are scheduled via Hangfire.

ActionParams key	Type	Description
`templateCode`	string	Notification template — determines subject, body, channel (email / in-app)
`recipientRole`	string	`SponsorContact`, `AssignedReviewer`, `StateAdmin`, `SupervisorLevel`, `AllStateStaff`
`delayDays`	int	0 = immediate. N = scheduled N days after state entry.
`suppressIfResolved`	bool	If true, delayed notification is cancelled if entity leaves state before delay expires

```
-- Immediate sponsor notification on claim approval\nWorkflowTransitionActions:\n  TransitionId  = [ApproveClaim: UnderReview -> Approved]\n  ActionCode    = 'SendNotification'\n  ExecutionOrder = 1\n  IsFederal     = false\n  ActionParams  = {
```

```
"templateCode": "ClaimApprovedSponsor", "recipientRole": "SponsorContact", "delayDays": 0
}\n\n-- Tenant override: Wisconsin adds director alert on LateClaimException
approval\nTenantTransitionOverrides:\n  TransitionId  = [ApproveLateClaimException:
PendingApproval -> Approved]\n  TenantCode    = 'WI'\n  OverrideType  = 'AddAction'\n
OverrideParams = {\n    "actionCode": "SendNotification",\n    "executionOrder": 99,\n
"actionParams": { "templateCode": "LateClaimDirectorAlert", "recipientRole":
"SupervisorLevel", "delayDays": 0 }\n  }
```

Timeout and Escalation: Scheduling Pattern

Timeouts are automatic transitions with `TriggerEvent = 'OnSchedule'` and a `DaysInStatePast` condition. A Hangfire job runs nightly and evaluates all pending automatic transitions.

```
-- Self-transition: escalate stale claim review after 10 days\nWorkflowTransitions:\n
EntityTypeId  = [Claim]\n  FromStateId   = [UnderReview]\n  ToStateId     = [UnderReview]   -
- self-transition: state unchanged\n  TransitionCode = 'EscalateStaleReview'\n  IsFederal
= false\n  IsAutomatic   = true\n  TriggerEvent  =
'OnSchedule'\n\nWorkflowTransitionConditions:\n  TransitionId  = [EscalateStaleReview]\n
ConditionCode = 'DaysInStatePast'\n  ConditionParams = { "thresholdDays": 10,
"businessDaysOnly": false }\n  IsFederal     = false\n\nWorkflowTransitionActions:\n
TransitionId  = [EscalateStaleReview]\n  ActionCode    = 'SendNotification'\n  ActionParams
= { "templateCode": "ReviewEscalationAlert", "recipientRole": "SupervisorLevel", "delayDays":
0 }\n  IsFederal     = false\n\n-- Tenant override: Hawaii uses 5 business days instead of 10
calendar days\nTenantTransitionOverrides:\n  TransitionId  = [EscalateStaleReview]\n
TenantCode    = 'HI'\n  OverrideType  = 'ReplaceCondition'\n  OverrideParams = {
"conditionCode": "DaysInStatePast", "conditionParams": { "thresholdDays": 5,
"businessDaysOnly": true } }
```

Supervisor Approval Routing: Override Pattern

Mechanism	When to Use	How
`TenantTransitionOverrides { OverrideType: 'Disable' }` + tenant-specific `WorkflowTransitions` rows	Inserting a new intermediate state (e.g., DirectorReview) that doesn't exist in the platform template	Disable the platform transition, add new states/transitions with TenantCode set
`TenantTransitionOverrides { OverrideType: 'AddCondition', conditionCode: 'HasRole' }`	Restricting an existing transition to a higher role without adding a new state	Add a HasRole condition with the required role code

```
-- Virginia: director approval required for LateClaimException\n-- Step 1: Disable default
PendingApproval -> Approved for VA\nTenantTransitionOverrides:\n  TransitionId  =
[LateClaimApprove: PendingApproval -> Approved]\n  TenantCode    = 'VA'\n  OverrideType  =
'Disable'\n\n-- Step 2: Add tenant-specific transitions inserting DirectorReview
```

```
state\nWorkflowTransitions (TenantCode = 'VA'):\n  PendingApproval -> DirectorReview
(IsFederal=false, TransitionCode='SendToDirector')\n  DirectorReview  -> Approved
(IsFederal=false, TransitionCode='DirectorApprove', Condition: HasRole='DivisionDirector')\n
DirectorReview  -> Denied          (IsFederal=false, TransitionCode='DirectorDeny')
```

Per-State Overrides

State	Override	OverrideType	Why it's safe
Maine	Auto-decline claims past 60-day late cutoff	AddCondition to SubmitClaim	SubmitClaim.IsFederal = false
Hawaii	Training module transitions enabled	Enable transitions in Training entity type	Training states are not federal
Indiana	DirectVer session transitions enabled (sponsor-facing DC resolution)	Enable DirectVer transitions for sponsor role	DirectVer states are not federal
Utah	Section 4/11 split enforced on FNS-10 report generation	AddCondition to GenerateFNSReport	Report generation transitions are not federal
All states	Cannot: remove Approved→Locked, remove FreezeClaimLaws action, bypass AllClaimErrorsResolved	Any attempt → GuardrailViolationException	All three have IsFederal = true

Executing a Transition

Engine execution sequence for a single transition request.

Steps 3 and 4 are the guardrail enforcement points. If a condition evaluation fails, the transition is rejected before any state change is written. The entity remains in its current state.

Actions are transactional. Steps 5 and 6 run inside a single database transaction. If `FreezeClaimLaws` fails, the status change in step 5 is rolled back. The claim stays in UnderReview. The `IsRollbackable` column flags which actions support compensation logic if a downstream action fails after they've already committed.

Professional Services Overrides: Workflow Config

Needs Lindsey and Courtney Review

Professional Services creates `TenantTransitionOverrides` rows through the Admin UI. Every write requires a justification field (minimum 20 characters). The guardrail check fires at write time.

OverrideType	What It Does	Guardrail Rejects If
`Disable`	Prevents this transition from appearing or firing for this tenant	WorkflowTransitions.IsFederal = true
`MakeAutomatic`	Changes a manual transition to fire automatically when conditions are met	WorkflowTransitions.IsFederal = true
`AddCondition`	Adds a new pre-condition to an existing transition	Target condition has IsFederal = true
`RemoveCondition`	Removes a non-federal pre-condition for this tenant	Target condition has IsFederal = true
`ReplaceCondition`	Swaps condition params for this tenant (e.g., change day threshold)	Target condition has IsFederal = true
`AddAction`	Adds a new action that fires on this transition for this tenant	Target action has IsFederal = true
`RemoveAction`	Removes a non-federal action for this tenant	Target action has IsFederal = true
`ChangeRole`	Changes which roles can trigger a transition for this tenant	WorkflowRolePermissions.IsFederal = true
`AddRole`	Adds an additional role that can trigger a transition	WorkflowTransitions.IsFederal = true

What PS Cannot Override:

- Disable federal transitions (Approved→Locked, DisburseClaim, etc.)

- Remove federal conditions (AllClaimErrorsResolved, NoOpenSeriousDeficiency, VCAChecklistPassed, ISPMeetsThreshold)
- Remove federal actions (FreezeClaimLaws, LockRecord, CreateEntitySnapshot, TriggerValidation, CreateDisbursementRecord)

Any attempt raises GuardrailViolationException and logs to AuditLogs with EntityType = 'FederalGuardrailViolation'

Audit Logs

Every write creates an AuditLogs row: TenantCode, ActorUserId, OverrideType, TransitionId, OverrideParams, **Justification** (required), timestamp.

Justification Requirement

Every write to `TenantTransitionOverrides` requires a justification. **The write service rejects the row if Justification is empty or fewer than 20 characters.**

Rule	Enforcement
Justification field cannot be empty	UI: Save button disabled until field has content. API: write service returns 400 if empty.
Minimum 20 characters	UI: character counter turns green at 20. API: write service validates length.
No template phrases	UI warning (not blocking) if justification matches a blocked phrase list ("N/A", "per request", "test", etc.)
Justification stored permanently	Written to AuditLogs in autonomous transaction — survives rollback of the override write itself.
Cannot be edited after save	AuditLogs is append-only. Corrections require a new entry.

Professional Services Roles

Role	Can	Cannot
`PS_Configurator`	Create and edit TenantTransitionOverrides.	Modify platform WorkflowStates or

	View audit log.	WorkflowTransitions rows. Delete audit records.
`PS_Lead`	All of PS_Configurator. Approve override changes before they activate when two-person rule is enabled.	Modify IsFederal flags. Bypass guardrail checks.
`PS_ReadOnly`	View workflow diagrams and override history.	Write anything.

Two-Person Rule (Optional per Tenant)

Tenants can require PS_Lead approval before an override activates. The PS_Lead who approves cannot be the same person who created the override. Recommended for tenants with nTIG contracts (AK, HI, ME, MO, UT, WI?) where workflow configuration errors could affect federal reporting or reimbursement cycles.

Two-Person Rule State	Behavior
Disabled (default)	Overrides written by PS_Configurator take effect immediately.
Enabled for tenant	Overrides staged. PS_Lead approves or rejects. Rejection reason stored in AuditLogs.
Who sets it	Engineering sets this flag per tenant in a migration. PS_Lead cannot self-enable the rule.

Workflow Admin UI

Professional Services configures tenant workflow behavior through the Admin page — an internal tool accessible only to LINQ staff. Changes take effect immediately; no deployment required.

Feature	What PS Does	Underlying Write
Visual workflow diagram	Views the state machine for a selected tenant and entity type. Federal states show lock icon and CFR citation.	Read-only

Enable / disable transitions	Toggles optional transitions on/off per tenant	TenantTransitionOverrides { OverrideType: 'Disable' }
Document gates	Adds or removes document type requirements on a transition	TenantTransitionOverrides { OverrideType: 'AddCondition' / 'RemoveCondition', conditionCode: 'AllRequiredDocsAttached' }
Notification rules	Adds, removes, or adjusts notification actions per transition	TenantTransitionOverrides { OverrideType: 'AddAction' / 'RemoveAction', actionCode: 'SendNotification' }
Escalation thresholds	Changes DaysInStatePast threshold for automatic escalation	TenantTransitionOverrides { OverrideType: 'ReplaceCondition', conditionCode: 'DaysInStatePast' }
Role permissions	Adds roles that can trigger a transition	TenantTransitionOverrides { OverrideType: 'AddRole' / 'ChangeRole' }
Audit log view	Sees all override history for a tenant with justifications	Read-only from AuditLogs

Allowed Configurations

Action	Example
Disable an optional transition	Montana doesn't use SupervisorReview on claims
Add a document gate	Alaska requires site affidavit before approval
Add a notification rule	Wisconsin alerts director on every late claim approval
Adjust escalation threshold	Hawaii escalates after 5 business days, not 10

Add a role to a transition	Maine allows Program Manager (not just Director) to approve exceptions

Blocked Actions

Action	Why blocked
Disable a federal transition	IsFederal = true — toggle hidden
Remove a federal condition	IsFederal = true — delete button hidden
Remove a federal action	IsFederal = true — delete button hidden
Edit IsFederal flag	Field not exposed anywhere in UI
Modify platform WorkflowStates rows	No write access to cnp_global schema tables

UI Screens: Examples

Examples generated by Claude. Not final. Needs UX Designer.

Guardrail Behavior in the UI

The UI does not allow PS to attempt a federal override. Elements are hidden or disabled before any API call is made. The UI suppression of controls is a usability feature, not security. **The actual enforcement is the API write service guardrail check and the PostgreSQL trigger.** Both fire regardless of how the write arrives.

Situation	UI Behavior	What PS Sees
Federal state node in diagram	Lock icon overlaid. No configuration panel opens on click.	Tooltip: "This state is federally required by [7 CFR citation]. It cannot be modified or disabled."
Federal transition edge in diagram	Red edge, lock icon. Click opens read-only panel only.	Panel shows conditions and actions with no edit/delete controls visible.
Federal condition in Document Gates tab	Delete button is not rendered for the row.	Row shows "🔒 Cannot remove" in the Remove column.

Federal action in Notifications tab	Delete button is not rendered for the row.	Row shows "🔒 Cannot remove" in the Remove column.
Disable Transition button on a federal transition	Button is not rendered in the panel.	Panel does not show a Disable option.
API-level guardrail fires despite UI protection	Error toast: "This configuration is federally protected and cannot be modified."	Error is logged to AuditLogs as FederalGuardrailViolation by the user's identity.

Schema Gap: StatusTransitionRules — Added July 2026

Identified during review of CTS stored procedures with Austin (June 30, 2026) and added to CNP_ERD.html. The CTS `status_get` stored procedure hard-codes valid next-status options per form type. This logic was missing from the greenfield design as an explicit table. Also see: **TenantErrorConfig** on the CNP Configuration Defaults page.

StatusTransitionRules cnp_global

CTS has a `status_get` stored procedure for each form type. It hard-codes which statuses are valid next steps from every current status — e.g., from *Submitted* a Claim can go to *Approved* or *Denied*, not back to *Incomplete*. This logic was missing from the greenfield design as a data table.

The `WorkflowTransitions` table handles this for the full engine (with conditions, actions, tenant overrides, and federal guardrails). `StatusTransitionRules` is the lightweight companion used for two distinct purposes: (1) UI queries — what buttons or options to show for this entity's current status; (2) API pre-validation of status change requests before the full engine runs, and for admin bulk operations that do not go through the engine.

Column	Notes
StatusTransitionId	PK
FormTypeCode	Application, Claim, Adjustment, AdminReview, etc. Maps to WorkflowEntityTypes.EntityTypeCode
FromStatusCode	FK → cnp_global status catalog (WorkflowStates.StateCode)

ToStatusCode	FK → cnp_global status catalog (WorkflowStates.StateCode)
IsManual	True = user-triggered transition (show a button). False = system-only (no UI affordance shown)
TenantCode	null = applies to all tenants. Non-null = tenant-specific rule, added by PS configuration, not migration

Relationship to WorkflowTransitions: Every StatusTransitionRules row has a corresponding WorkflowTransitions row. StatusTransitionRules is the fast read for what actions are available from a given state; WorkflowTransitions is the execution record with conditions, actions, and guardrails. A migration that adds a new transition adds a row to both. They stay in sync by convention — StatusTransitionRules rows are never written without a corresponding WorkflowTransitions row.